DrillBit

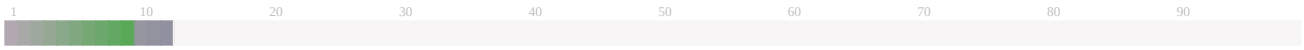
The Report is Generated by DrillBit Plagiarism Detection Software

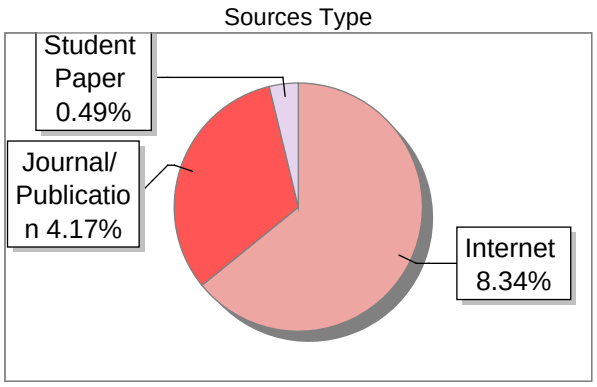
### *Submission Information*

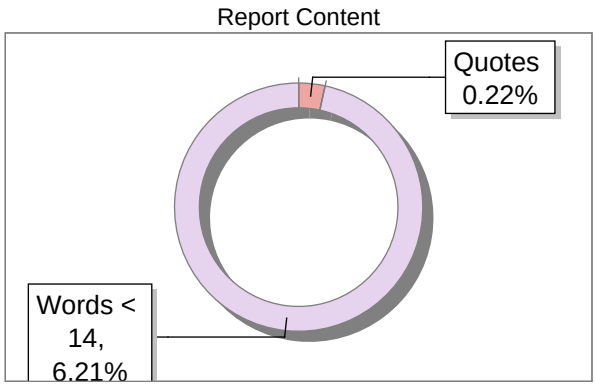
| | |
|---|---|
| Author Name | Ms Lincy |
| Title | Mini Project |
| Paper/Submission ID | 3736890 |
| Submitted by | dr.kavithat.nhce@newhorizonindia.edu |
| Submission Date | 2025-06-05 14:07:01 |
| Total Pages, Total Words | 64, 10709 |
| Document type | Project Work |

### *Result Information*

Similarity **13 %**




### *Exclude Information*

| | |
|---|---|
| Quotes | Not Excluded |
| References/Bibliography | Not Excluded |
| Source: Excluded < 14 Words | Not Excluded |
| Excluded Source | **0 %** |
| Excluded Phrases | Not Excluded |

### *Database Selection*

| | |
|---|---|
| Language | English |
| Student Papers | Yes |
| Journals & publishers | Yes |
| Internet or Web | Yes |
| Institution Repository | Yes |

A Unique QR Code use to View/Download/Share Pdf File

DrillBit

**DrillBit Similarity Report**

| 13 | 80 | B |
|---|---|---|
| SIMILARITY % | MATCHED SOURCES | GRADE |

**A-Satisfactory (0-10%)**
**B-Upgrade (11-40%)**
**C-Poor (41-60%)**
**D-Unacceptable (61-100%)**

| LOCATION | MATCHED DOMAIN | % | SOURCE TYPE |
|---|---|---|---|
| 1 | www.mygreatlearning.com | 2 | Internet Data |
| 2 | psscive.ac.in | <1 | Publication |
| 3 | fastercapital.com | <1 | Internet Data |
| 4 | www.goglides.dev | <1 | Internet Data |
| 5 | fastercapital.com | <1 | Internet Data |
| 6 | moam.info | <1 | Internet Data |
| 7 | e-journal.uajy.ac.id | <1 | Publication |
| 8 | technodocbox.com | <1 | Internet Data |
| 9 | www.ncbi.nlm.nih.gov | <1 | Internet Data |
| 10 | vignan.ac.in | <1 | Publication |
| 11 | www.mdpi.com | <1 | Internet Data |
| 12 | REPOSITORY - Submitted to Exam section VTU on 2024-07-31 15-47 915740 | <1 | Student Paper |
| 13 | Encounters between two sympatric carnivores red foxes (Vulpes vulpes) and Europ by D-2004 | <1 | Publication |

| | | | |
|---|---|---|---|
| 14 | qdoc.tips | <1 | Internet Data |
| 15 | Anonymous voting for multi-dimensional CV quantum system by Shi-2016 | <1 | Publication |
| 16 | odr.chalmers.se | <1 | Publication |
| 17 | 1library.co | <1 | Internet Data |
| 18 | www.freepatentsonline.com | <1 | Internet Data |
| 19 | 123dok.com | <1 | Internet Data |
| 20 | l1z2g9.github.io | <1 | Internet Data |
| 21 | www.osce.org | <1 | Publication |
| 22 | canvas.uw.edu | <1 | Publication |
| 23 | www.thecommonsjournal.org | <1 | Publication |
| 24 | docplayer.net | <1 | Internet Data |
| 25 | fastercapital.com | <1 | Internet Data |
| 26 | pdfcookie.com | <1 | Internet Data |
| 27 | REPOSITORY - Submitted to Bannari Amman Institute of Technology on 2023-02-21 13-48 | <1 | Student Paper |
| 28 | gisuser.com | <1 | Internet Data |
| 29 | qdoc.tips | <1 | Internet Data |
| 30 | www.assureuas.org | <1 | Publication |
| 31 | Learning tactical human behavior through observation of human performance by Fernlund-2006 | <1 | Publication |

| | | | |
|---|---|---|---|
| 32 | moam.info | <1 | Internet Data |
| 33 | qdoc.tips | <1 | Internet Data |
| 34 | www.guvi.in | <1 | Internet Data |
| 35 | www.vivekanandacollege.edu.in | <1 | Publication |
| 36 | iitk.ac.in | <1 | Internet Data |
| 37 | Submitted to U-Next Learning on 2025-02-01 20-18 3234637 | <1 | Student Paper |
| 38 | Trading Votes for Votes A Laboratory Study by Casella-2020 | <1 | Publication |
| 39 | www.itconvergence.com | <1 | Internet Data |
| 40 | YOLOv8-Enabled Real-Time Crop Health Monitoring with Conversational Diagnosis a By, Jampana Venkata Arjun Varma, Yr-2025,5,12 | <1 | Publication |
| 41 | apacwomen.ac.in | <1 | Publication |
| 42 | A Randomized-Controlled Trial of EMDR Flash Technique on Traumatic Symptoms, De By Alian Burak Yaar, Emre Konu, Yr-2022,3,24 | <1 | Publication |
| 43 | translate.google.com | <1 | Internet Data |
| 44 | www.americanprogress.org | <1 | Internet Data |
| 45 | www.ezoic.com | <1 | Internet Data |
| 46 | ejurnal.undana.ac.id | <1 | Publication |
| 47 | Extended Development of a Fission Gas Release Behavior Model Inside Spherical F By Jingyu Guo, Songbai Cheng, Ka, Yr-2023,9,18 | <1 | Publication |
| 48 | fastercapital.com | <1 | Internet Data |
| 49 | instapage.com | <1 | Internet Data |

| | | | |
|---|---|---|---|
| 50 | interactions.tn | <1 | Internet Data |
| 51 | lup.lub.lu.se | <1 | Publication |
| 52 | moam.info | <1 | Internet Data |
| 53 | moam.info | <1 | Internet Data |
| 54 | strategicstudyindia.com | <1 | Internet Data |
| 55 | videos.jeydancepilates-formations.fr | <1 | Internet Data |
| 56 | www.freepatentsonline.com | <1 | Internet Data |
| 57 | www.geeksforgeeks.org | <1 | Internet Data |
| 58 | www.turing.com | <1 | Internet Data |
| 59 | aceproject.org | <1 | Internet Data |
| 60 | apps.dtic.mil | <1 | Publication |
| 61 | Auricular pseudocysts A treatment with the Chulalongkorn University vacuum devi by Pakpoo-2001 | <1 | Publication |
| 62 | A Fully Decentralized Infrastructure for Subscription-based IoT Data Trading by Lin-2020 | <1 | Publication |
| 63 | A privacy-preserving cryptosystem for IoT E-healthcare by Hamza-2019 | <1 | Publication |
| 64 | Bridging communication gaps An AI-powered real-time system for sign language, By Naga Sasha Lakshmi Pokanati,, Yr-2025,3,30 | <1 | Publication |
| 65 | digitalcommons.pepperdine.edu | <1 | Publication |
| 66 | docobook.com | <1 | Internet Data |
| 67 | en.wikipedia.org | <1 | Internet Data |

| | | | |
|---|---|---|---|
| 68 | engessays.com | <1 | Internet Data |
| 69 | journal.unnes.ac.idsju | <1 | Internet Data |
| 70 | moam.info | <1 | Internet Data |
| 71 | Thesis Submitted to Shodhganga Repository | <1 | Publication |
| 72 | Two Approaches to Genre Analysis Three Genres in Modern American Engl by Xiao-2005 | <1 | Publication |
| 73 | Untangling the relationships among growth, profitability and survival in new fir by Delmar-2013 | <1 | Publication |
| 74 | via.library.depaul.edu | <1 | Publication |
| 75 | www.aiscreen.io | <1 | Internet Data |
| 76 | www.cbit.ac.in | <1 | Publication |
| 77 | www.network.bepress.com | <1 | Publication |
| 78 | www.prepbytes.com | <1 | Internet Data |
| 79 | www.w3.org | <1 | Internet Data |
| 80 | IEEE 2013 9th International Conference on Natural Computation (ICNC) by | <1 | Publication |

# CHAPTER 1

# INTRODUCTION

Voting is an essential process in any democratic setup, enabling individuals to make collective decisions and choose their representatives. Traditional paper-based voting methods often face challenges such as long queues, manual counting errors, and potential for duplication or fraud. To overcome these issues, this project introduces a digital Voting System designed to conduct elections in a secure, efficient, and transparent manner. This system allows registered users to participate in elections electronically from a secure environment. It is designed with a user-friendly interface and a robust backend to handle data storage, authentication, and vote tracking. The application includes role-based access where only authorized users can manage the system or cast a vote.

## 1.1. PROBLEM DEFINATION

The aim of this project is to develop a secure, efficient, and user-friendly electronic Voting System that simplifies the election process by digitizing the tasks of voter registration, candidate management, vote casting, and result generation. This system is designed to ensure fair and transparent elections, eliminate the issues of manual voting methods such as delays, errors, and duplication, enable one-person-one-vote functionality, provide realtime result updates make the election process faster, more organized, and accessible. By achieving these goals, the project seeks to create a reliable digital platform suitable for use in schools, colleges, organizations, and small communities.

## 1.2. OBJECTIVES

1.To provide a digital platform for conducting elections that is secure, accurate, and reliable.

2. To simplify the voter and candidate management process through an easy-to-use interface.

3. To ensure each voter can vote only once, maintaining the integrity of the election.
4. To allow administrators to monitor, manage, and analyse election results in realtime.
5. To reduce human error and prevent fraudulent activities that are common in manual voting systems.
6. To speed up the voting and result declaration process, making elections more efficient.
7. To improve accessibility and participation by making the voting process more convenient for users.

## 1.3. METHODOLOGY TO BE FOLLOWED

- User Registration: Voter and admin accounts created and verified.
- Authentication: Login system with secure password storage.
- Voting Process: One-time voting per user, data stored securely.
- Vote Counting: Automated result calculation using SQL queries.
- Admin Controls: Manage elections, approve voters, view reports.
- Data Encryption: Ensures secure transmission of sensitive information.
- Multi-Factor Authentication: Adds extra security for login.
- Role-Based Access: Different functionalities for voters and administrators.
- Error Handling Mechanism: Prevents system failures during elections.

## 1.4. EXPECTED OUTCOMES

The online voting system aims to achieve a set of key outcomes that ensure a secure, reliable, and efficient voting experience for both voters and administrators. The anticipated results include:

- **Secure Authentication Mechanism**: T
  - he system will offer a robust and secure login process for both voters and administrators, minimizing the risk of unauthorized access and ensuring that only registered individuals can participate in the election process.
  - 
- **Efficient Data Management**:
  - All voter and candidate-related information will be systematically organized and stored using a reliable database system. This will ensure quick retrieval, data consistency, and seamless management during the election process.
  - 
- **User-Friendly Interface**:
  - A simple, intuitive, and responsive electronic voting platform will be developed. This will make the voting process easy to navigate, even for users with minimal technical knowledge.
  - 
- **One Vote Per Voter Enforcement**:
  - The system will incorporate mechanisms to strictly enforce the rule that each registered voter is allowed to cast only one vote. This feature is critical for maintaining the integrity and fairness of the election.
  - 
- **Real-Time Vote Tracking and Instant Results**:
  - As votes are cast, the system will automatically update and display live vote counts. Final results will be made available immediately after the voting window closes, enhancing transparency and efficiency.

## 1.5. HARDWARE AND SOFTWARE REQUIREMENTS

To successfully implement and run the online voting system, the following hardware and software components are required:

### Hardware Requirements

- **Processor**: Intel Pentium i3 or higher (to ensure smooth performance).
- **Memory (RAM)**: At least 4 GB RAM to handle multiple user requests simultaneously.
- **Storage**: Minimum of 500 GB hard drive space to store application files and database records.
- **Monitor**: 15-inch LED display or larger for optimal user experience during development and monitoring.

### Software Requirements

- **Operating System**: Microsoft Windows 10 or Windows 11 for compatibility with the development environment and tools.
- **Programming Language**: Java will serve as the core language for backend development, ensuring cross-platform support and strong object-oriented capabilities.
- **Frontend Technologies**:
  - **JSP (JavaServer Pages)** – For dynamic web content generation.
  - **HTML, CSS, JavaScript** – For designing a responsive and user-friendly interface.
- **Database Management System**: **MySQL** will be used to manage all the data related to voters, candidates, and election results in a secure and organized manner.

# CHAPTER 2

# FUNDAMENTALS OF JAVA

## 2.1. INTRODUCTION TO JAVA

Java is an object-oriented programming language developed by Sun Microsystems in 1995. It is platform-independent in nature because of the JVM or "Write Once Run Anywhere" philosophy. Java is widely used from web applications to mobile apps (especially Android) to enterprise software.

*Fig. No. 2.1 Different versions of Java over the years*

## 2.2. ADVANTAGES OF JAVA

Java is a platform-independent and secure programming language. Multi-threaded Java supports built-in networking which opens it to numerous usable applications. Strong memory management with automatic garbage collection further proves its efficiency.

## 2.3. DATA TYPES

Data types are classified into two, which are primitive and non-primitive. Predefined Java data types such as int, float, and char fall under primitive types. The nonprimitive. consists of Arrays, Strings, and Objects. Java is strongly typed which guarantees type safety during compilation.

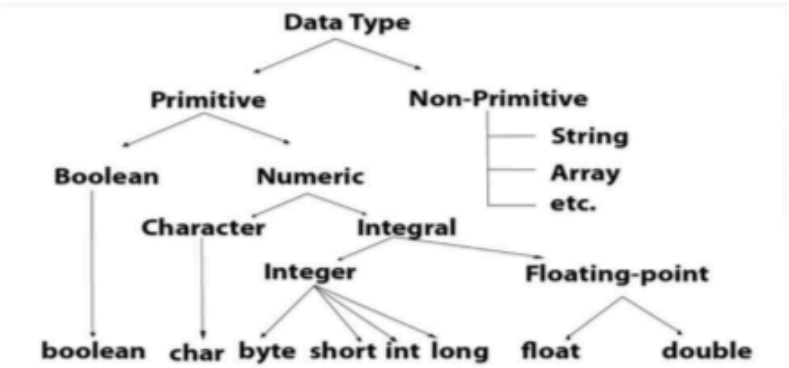



*Fig. No. 2.2 Data types in java*

## 2.4. CONTROL FLOW

Control flow statements govern the order of instruction execution. Thus, these statements help develop logical and efficient instruction sequences. They include conditional statements (if, if-else, switch), loops (for, while, do-while), and even branching statements (break, continue, return).

## 2.5. METHODS

Java methods are code segments that are intended to carry out a specific function and enable reusability of code. A method possesses a name, return type, and can take parameters. Java allows both static and instance methods, and methods may be overloaded (same name with different parameters).

## 2.6. OBJECT ORIENTED CONCEPTS

Java is founded upon four principal object-oriented concepts: encapsulation, inheritance, polymorphism, and abstraction. These concepts enable programmers to represent realworld situations, encourage code reusability, and improve maintainability and scalability. Java disciplines OOP using classes and objects.

## 2.7. EXCEPTION HANDLING

Java offers a powerful mechanism for processing runtime exceptions using try, catch, finally, and throw/throws keywords. Exception processing ensures that software programs are capable of handling unanticipated problems with more reliability. Java offers checked as well as unchecked exceptions.

## 2.8. FILE HANDLING

Java provides I/O classes within the java.io and java.nio packages to read and write information from files. Typical classes are File, FileReader, BufferedReader, and FileWriter. File handling is essential for programs that need persistent storage or data transfer.

## 2.9. PACKAGES AND IMPORT

Packages in Java are employed to collect related classes and interfaces, enhancing code modularity and organization. Java includes several built-in packages (such as java.security, java.net), and programmers can define their own packages. The import statement is employed to import classes from other packages.

## 2.10. INTERFACES

A Java interface is a reference type that is used to specify a contract for classes to follow. It has abstract methods by default (Java 8 and above supports default and static methods as well). Interfaces support multiple inheritance and assist in loose coupling in applications.

## 2.11. CONCURRENCY

Concurrency in Java enables multiple threads to run concurrently, enhancing application performance. Java offers the Thread class and Runnable interface to create threads. The java.util.concurrent package provides utilities for coordinating complex concurrent tasks and thread safety.

# CHAPTER 3

# JAVA COLLECTIONS GUIDE

## 3.1. JAVA COLLECTIONS OVERVIEW

Java Collections Framework (JCF) is a consistent architecture for storing and operating on collections of objects. It consists of interfaces such as List, Set, Map, and Queue, as well as their implementations. Collections make data handling easier using pre-existing data structures and algorithms.

## 3.2. JAVA DEVELOPMENT IMPORTANCE

Collections are important for handling dynamic data in a Java application in an efficient manner. Collections avoid the need for implementing custom data structures, thereby saving time. Collections also enable sorting, searching, and iteration, which are typical needs of enterprise and real-world applications.

## 3.3. BENEFITS AND USE CASES

Java collections enhance the efficiency, flexibility, and readability of code through the provision of standard, reusable building blocks. They're prevalent in data manipulation, caching, searching, and queuing. Collections also interoperate effectively with Java's concurrency and generics facilities for scalable type-safe programming.

## 3.4. CORE COLLECTION INTERFACES

The central interfaces in the Collections Framework are Collection, List, Set, Queue, and Map. Each one has a specific function: List permits ordered elements, Set guarantees uniqueness, Map stores key-value pairs, and Queue facilitates FIFO ordering.

## 3.5. COMMON COLLECTION IMPLEMENTATIONS

Common implementations are ArrayList, LinkedList, HashSet, TreeSet, HashMap, and TreeMap. They all have certain performance attributes—for example, ArrayList is efficient for random access, but LinkedList is good at insertions and deletions.

## 3.6. ITERATORS AND COLLECTIONS API

Iterators offer a mechanism to step through collections one element at a time. The Iterator and ListIterator interfaces provide methods such as hasNext() and next(). The Collections API also contains utility classes such as Collections and Arrays for generalpurpose tasks such as sorting and searching

## 3.7. CUSTOM COLLECTIONS AND GENERICS

Java enables programmers to write specialized collection classes for special requirements, usually by subclassing an existing class or implementing an interface. Generics enable collections to hold objects of a certain type, enhancing type safety and minimizing type casting.

# CHAPTER 4

# FUNDAMENTALS OF DBMS

## 4.1. INTRODUCTION

A Database Management System, or DBMS, is a program that controls the data and offers an interface to allow users to interact with the database. It makes sure the data is stored, retrieved, and handled efficiently. DBMS hides the complexity of data storage and provides data manipulation tools and administration. MySQL, Oracle, and PostgreSQL are examples.

## 4.2. CHARACTERISTICS OF DBMS

BMS offers data abstraction, data independence, and effective data access. It allows multiple user environments as well as data security and integrity enforcement. Other important features are concurrency control, backup and recovery, and minimized data redundancy. These features enhance DBMS compared to legacy file systems.


*Fig. No. 4.1 Characteristics of DBMS*

## 4.3. DATA MODEL

A data model specifies the structure, storage, and manipulation of data in a DBMS. It serves as a template for database design. Some popular data models are hierarchical, network, relational, and object-oriented models. The most popular one is the relational model, which presents data in the form of tables (relations).

## 4.4. THREE-SCHEMA ARCHITECTURE

This structure isolates the database into three levels: internal, conceptual, and external. The internal level is concerned with physical storage, the conceptual level is concerned with describing the logical structure, and the external level gives user-specific views. This isolation enhances data abstraction and independence.

## 4.5. COMPONENT MODULES OF DBMS

DBMS consists of some essential components: the Query Processor, Storage Manager, Transaction Manager, and Metadata Manager. They manage query execution, storage of data, transaction management, and schema definitions. Collectively, they facilitate efficient and safe data operations.

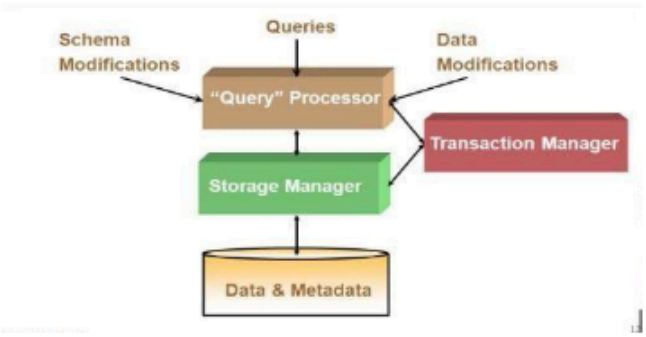



*Fig. No. 4.2 Component Modules of DBMS*

## 4.6. ENTITY-RELATIONSHIP (ER) MODEL

The ER model graphically depicts data and their interrelations in terms of entities, attributes, and relationships. Entities are things (such as Student or Course), attributes define them, and relationships indicate connections (such as Enrolled). ER diagrams assist in database design through a clear data structure.

## 4.7. RELATIONAL SCHEMA

A relational schema describes the organization of a relational database in terms of tables, with every table standing for an entity or relationship. The schema comprises table names, column names, and types. The schema also states primary and foreign keys to set relationships among tables, while maintaining data consistency.

# CHAPTER 5

# FUNDAMENTALS OF SQL

## 5.1. INTRODUCTION

SQL (Structured Query Language) is a standard language to manage and manipulate relational databases. SQL allows users to query, update, and manage data. SQL is popularly implemented across DBMS platforms such as MySQL, Oracle, and SQL Server. SQL is powerful as it can perform intricate data operations and provides database integrity and consistency.

## 5.2. SQL COMMANDS

SQL statements are classified into a number of categories depending on the function they carry out. They include Data Definition Language (DDL), Data Manipulation Language (DML), Data Control Language (DCL), Transaction Control Language (TCL), and Data Query Language (DQL). Every category of statement assists in controlling various aspects of database operations, including structure definition, data retrieval, and transaction control.

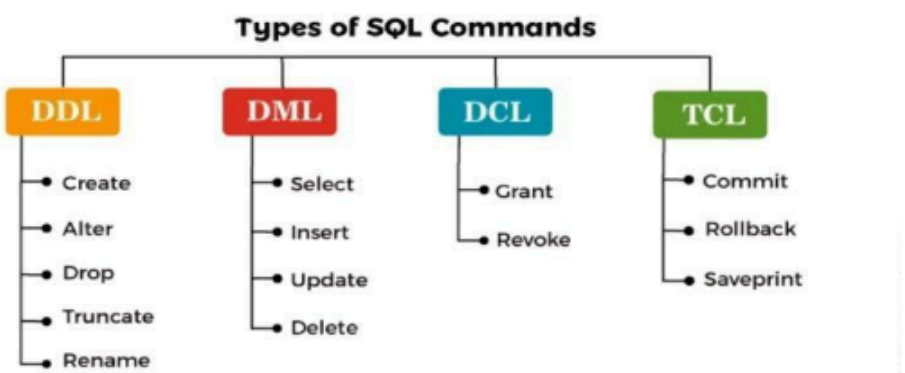



*Fig. No. 5.1 Types of SQL Commands*

## 5.3. DATA DEFINITION LANGUAGE (DDL)

Data Definition Language (DDL) is a set of SQL commands used to define and manage the structure of database schema objects such as tables, views, indexes, and schemas. These commands primarily focus on the creation and modification of database structures, rather than handling the data within them. The main DDL commands include:

- **CREATE**: This command is used to create new database objects like tables, schemas, indexes, or views. For example, CREATE TABLE is used to define the columns, data types, and constraints of a new table.
- **ALTER**: This command is employed to modify the structure of existing database objects. It allows for operations such as adding or deleting columns, changing data types, and renaming fields.
- **DROP**: The DROP command permanently deletes a database object and all its contents. For instance, dropping a table will remove both the structure and all the data it contains.
- **TRUNCATE**: While similar to DELETE, this command removes all the records from a table without deleting the table itself. The table structure remains intact, allowing it to be reused.

DDL operations directly affect the database schema and are generally **auto-committed**, meaning that the changes are instantly applied and cannot be rolled back. This makes them powerful, but it also requires caution during use to avoid unintentional data loss or structure corruption.

## 5.4. DATA MANIPULATION LANGUAGE (DML)

Data Manipulation Language (DML) consists of SQL commands used to manipulate the actual data stored within database tables. These commands allow users to insert new

data, modify existing records, or remove data from tables. The primary DML commands include:

- **INSERT**: Used to add one or more new records into a table. It specifies the target table and the values to be inserted into specific columns.
- **UPDATE**: Allows for modification of existing data in a table. The UPDATE command is typically used with a WHERE clause to specify which records should be updated.
- **DELETE**: Removes one or more records from a table based on a condition. If the WHERE clause is omitted, all records will be deleted.

Unlike DDL commands, DML operations are **not auto-committed**. This means changes made using DML need to be explicitly saved using the **COMMIT** command or undone using **ROLLBACK** if necessary. This control allows developers to review and verify changes before finalizing them, which is especially useful in complex transactions and data correction processes.

## 5.5. DATA CONTROL LANGUAGE (DCL)

Data Control Language (DCL) is a subset of SQL used to control access and permissions within the database system. It plays a critical role in database security by determining who can access or manipulate specific data and objects. The two main DCL commands are:

- **GRANT**: This command provides specific privileges to users or roles. For example, a user can be granted permission to perform actions such as SELECT, INSERT, UPDATE, or DELETE on a particular table.
- **REVOKE**: This command is used to remove or restrict previously granted permissions from a user or role. It helps ensure that access rights are updated as user roles or responsibilities change.

DCL commands are typically executed by database administrators to enforce security protocols and ensure that only authorized users have access to sensitive data. Proper use

of DCL enhances the confidentiality, integrity, and availability of the data in systems where multiple users interact with the database.

## 5.6. TRANSACTION CONTROL LANGUAGE (TCL)

Transaction Control Language (TCL) deals with the management of transactions in a database system. A transaction is a sequence of operations performed as a single logical unit of work, and TCL commands help ensure data consistency and integrity throughout this process. The key TCL commands include:

- **COMMIT**: This command permanently saves all changes made during the current transaction. Once committed, the changes become visible to other users and cannot be undone.
- **ROLLBACK**: Reverts changes made during the current transaction to the last committed state. It is useful for undoing errors or inconsistencies in data operations.
- **SAVEPOINT**: Allows the creation of intermediate points within a transaction. You can roll back to a specific savepoint without undoing the entire transaction, which adds flexibility to error recovery.
- **SET TRANSACTION**: Used to set specific properties for a transaction, such as the isolation level, which controls how data changes made by one transaction are visible to others.

TCL commands are essential in multi-user environments where multiple transactions might be running simultaneously. They help maintain the **ACID (Atomicity, Consistency, Isolation, Durability)** properties of a database, which are crucial for reliable transaction processing.

## 5.7. DATA QUERY LANGUAGE (DQL)

Data Query Language (DQL) is centered around data retrieval operations. The **SELECT** command is the primary DQL command and is used to fetch data from one or more tables in a database. DQL enables users to extract meaningful information based on specific conditions and filters. Key features of DQL include:

- **SELECT**: Retrieves specific columns or all data (SELECT *) from a table. It can be used with conditions to filter the output using the **WHERE** clause.
- **ORDER BY**: Sorts the result set based on one or more columns in ascending or descending order.
- **GROUP BY**: Groups rows that have the same values in specified columns into summary rows, often used with aggregate functions.
- **HAVING**: Works like the WHERE clause but is used to filter grouped data, especially when aggregate functions are involved.
- **JOINS**: Combines rows from two or more tables based on a related column. Common types include INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.
- **Aggregate Functions**: Functions like COUNT, SUM, AVG, MAX, and MIN are used to perform calculations on data and summarize results.

DQL is vital for reporting, data analysis, and visualization in applications such as an online voting system, where insights like total votes per candidate, voter turnout, and demographic statistics may be required.

# CHAPTER 6

## DESIGN AND ARCHITECTURE

**1. Modularity:** The system should be designed with distinct components such as user authentication, ballot management, vote casting, and result tabulation. This separation allows for easier updates and maintenance.

**2.Scalability:** The architecture must support a growing number of users, votes, and election types without requiring significant redesign. This ensures that the system can handle peak loads during major elections.

**3.Maintainability:** The codebase should be structured for easy updates, testing, and debugging. Clear separation of concerns will facilitate long-term manageability and allow for quick adaptations to changing regulations or requirements.

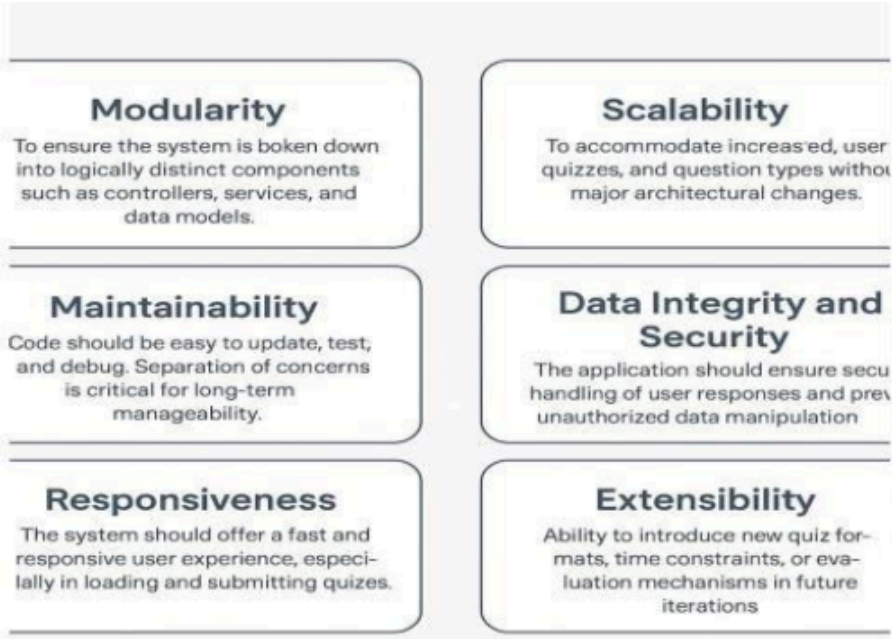
**4.Responsiveness:** The user interface should provide a fast and intuitive experience, ensuring that voters can easily navigate the system, cast their votes, and receive confirmation without delays

**5.Data Integrity and Security:** The system must ensure the secure handling of voter data and responses, employing encryption and secure protocols to prevent unauthorized access and data manipulation. Voter anonymity must also be preserved.

**6.Extensibility:** The design should allow for the introduction of new voting methods, such as ranked-choice voting or multi-candidate elections, as well as the ability to implement new features like real-time result tracking or enhanced voter verification processes.

**7.Auditability:** The system should provide mechanisms for auditing and verifying votes to ensure transparency and trust in the electoral process. This includes maintaining logs of all actions taken within the system.

**8.User Accessibility:** The voting platform should be accessible to all eligible voters, including those with disabilities Compliance with accessibility standards is essential.




## 6.2. Database Structure

The database design for the online voting system follows a relational model to ensure normalized and consistent storage of voting-related data. Key entities and their attributes include:

The database design for the online voting system is grounded in a relational model, which ensures that data is stored in a normalized and consistent manner. This approach is crucial for maintaining data integrity, facilitating efficient data retrieval, and supporting the overall functionality of the voting system. The database comprises several key entities, each serving a specific purpose in the voting process.

**Voter Table:** At the core of the system is the **Voter Table**, which is designed to store essential information about each voter. Each voter is assigned a unique identifier (id), which serves as the primary key for the table. In addition to the unique identifier, the table includes fields for the voter's full name, email address, and a securely hashed password for authentication purposes. The inclusion of a verification status flag is also critical, as it indicates whether the voter has completed the necessary steps to verify their

identity before participating in an election. This table is fundamental to ensuring that only eligible voters can cast their votes, thereby enhancing the security and integrity of the electoral process.

**Election Table:** Complementing the Voter Table is the **Election Table**, which captures vital details about each election conducted through the system. This table also features a unique identifier (id) as its primary key, along with descriptive fields such as the title of the election, the start and end dates, and the current status of the election (e.g., scheduled, ongoing, or completed). By maintaining a comprehensive record of elections, the system can efficiently manage multiple electoral events, allowing for seamless transitions between different voting periods and ensuring that voters are informed about upcoming elections.

**Candidate Table:** The Candidate Table plays a crucial role in linking candidates to their respective elections. Each candidate is assigned a unique identifier (id), and the table includes fields for the candidate's name and political party affiliation. A foreign key relationship is established with the Election Table, allowing the system to associate each candidate with the specific election in which they are participating. This structure not only facilitates the management of candidate information but also enables voters to easily access details about the candidates they can vote for during an election.

**Vote Table:** The Vote Table is essential for recording the votes cast by voters. Each entry in this table is assigned a unique identifier (id) and includes foreign keys that reference both the voter and the candidate associated with the vote. Additionally, a timestamp field is included to track when each vote was submitted. This table is critical for ensuring that the voting process is accurately recorded and that each voter's choice is securely linked to the corresponding candidate.

**Audit Log Table:** To enhance transparency and accountability within the voting system, an **Audit Log Table** is implemented. This table maintains a comprehensive record of actions taken within the system, such as votes cast, voter registrations, and any administrative actions performed by election officials. Each log entry is assigned a unique identifier (id) and includes fields for the action description, timestamp, and a reference to

the voter associated with the action. The audit log serves as a vital tool for monitoring the integrity of the voting process and provides a mechanism for verifying that all actions are properly documented.

Overall, the database structure of the online voting system is designed to promote flexibility, security, and efficiency in managing the electoral process. By utilizing a relational model, the system can effectively handle various elections, candidates, and voter interactions while ensuring that data integrity is maintained throughout. This comprehensive approach not only supports the operational needs of the voting system but also fosters trust and confidence among voters, ultimately contributing to a more democratic and transparent electoral process.

## 6.2. High Level Architecture

The online voting system adopts a three-tier architecture, which is well-suited for webbased applications. This architecture promotes a clear separation of concerns, enhancing maintainability, scalability, and security.

**1. Presentation Layer (Frontend):**

The **Presentation Layer** of the online voting system serves as the interface between the end-users (voters and administrators) and the underlying system functionalities. It is built using core web technologies—**HTML**, **CSS**, and **JavaScript**—which together enable the development of a responsive, intuitive, and interactive user experience.

This layer plays a vital role in ensuring that the platform is not only functional but also user-friendly and accessible across a wide range of devices and screen sizes. The objective is to provide a seamless interface through which users can engage with the system effortlessly, regardless of their technical background or device capability.

*Technologies Used*

- **HTML (HyperText Markup Language)**: Acts as the structural foundation of all web pages in the system. It defines the layout of the content, such as text, input forms, buttons, images, and hyperlinks.
- **CSS (Cascading Style Sheets)**: Responsible for the visual presentation of HTML elements. CSS styles are used to enhance the aesthetics of the interface, including colors, fonts, spacing, and alignment. It also supports media queries for implementing responsive design.
- **JavaScript**: Adds interactivity and client-side functionality to the application. It is used to validate forms, manage UI behavior (such as hiding or showing sections dynamically), handle user interactions (e.g., button clicks), and make asynchronous requests to the server (using AJAX or fetch APIs) for real-time updates like displaying live election results.

*Key Interfaces*

- The entry point for users visiting the online voting portal. This page typically provides an overview of the platform, guidance on how to vote, links to login or register, and information about upcoming elections. It is designed to be welcoming and easy to navigate, encouraging voter participation.
- This interface is central to the voting process. Once authenticated, voters are redirected to this page where they can view candidate profiles, read election instructions, and cast their vote. JavaScript ensures that users cannot vote more than once and dynamically updates the voting form based on the selected election. User feedback and confirmation messages are displayed upon vote submission to assure voters of successful participation.

Other supplementary interfaces might include:

- **login.html** for user authentication.

- **register.html** for new voter sign-up.
- **results.html** to display live or final election results.
- **admin.html** for election administrators to manage elections, candidates, and users.

*User Experience & Accessibility*

The presentation layer is designed with **responsive design principles**, ensuring that the platform functions smoothly across a variety of devices, including desktops, laptops, tablets, and smartphones. This is achieved through:

- **Flexible grid layouts** and **media queries** to adapt the layout based on screen size.
- **Accessible navigation** with clearly labeled buttons and sections.
- **Optimized loading times** to maintain performance and usability, even in low-bandwidth environments.

The focus on **user-centric design** enhances usability and ensures that even users with minimal technical knowledge can navigate the platform effectively. Visual cues, form validation, error messages, and confirmation alerts are implemented to guide the user through each step of the process.

*Importance of the Presentation Layer*

In the context of an online voting system, the presentation layer is especially important because:

- It shapes the **first impression** for users, which affects trust and confidence in the platform.
- It directly impacts **voter engagement**—a clean, intuitive interface encourages more users to participate in the election.
- It serves as the medium through which critical tasks like **casting a vote**, **viewing candidates**, or **checking results** are executed.

Any lag, confusion, or poor design at this layer can hinder participation, reduce satisfaction, or even introduce user errors that compromise the fairness of the process.

**2. Business Logic Layer (Backend):**

The **Business Logic Layer (BLL)**, also known as the **application layer**, is the core of the online voting system. It encapsulates the essential functionalities and rules that govern how data is processed and how operations are performed throughout the platform. This layer is implemented using **Java** in conjunction with the **Spring Boot** framework, which offers a powerful and lightweight framework for building scalable, modular, and production-ready enterprise applications.

Spring Boot simplifies the setup and development of Java-based applications by providing built-in support for dependency injection, RESTful API development, data access integration, and application configuration, all of which are crucial in maintaining the structure and scalability of a system as sensitive and dynamic as an online voting platform.

**Role of the Business Logic Layer**

The business logic layer acts as an intermediary between the **presentation layer (frontend)** and the **data access layer (database)**. It is responsible for interpreting user inputs from the frontend, applying the necessary business rules, interacting with the database to perform operations, and returning results to the client interface. It ensures that the system behaves consistently and securely according to predefined rules and regulations.

This layer is vital in enforcing **electoral policies**, such as:

- Allowing only **authenticated and verified** users to vote.
- Ensuring **one vote per voter** per election.
- Managing **candidate eligibility** and election configurations.
- **Tabulating results** in real-time while maintaining data integrity and audit trails.

***Key Components***

Within the business logic layer, the system is divided into multiple components following the Model-View-Controller (MVC) architecture. The main components include:

**1. Controllers**

Controllers are entry points for user actions, often exposed via RESTful APIs. In this system, key controllers include:

- **ElectionController.java**:
  Handles HTTP requests related to elections. This includes retrieving upcoming elections, displaying election details, managing election creation and scheduling (admin-level), and updating election statuses (e.g., opening or closing the voting window).
- **VoteController.java**:
  Manages vote-related operations such as casting a vote, validating if the voter is eligible to vote, checking if a vote has already been cast, and retrieving voting history or statistics. It communicates with service classes to apply business rules before interacting with the database.

**2. Services**

Although not explicitly shown, service classes are implied to exist and represent the **true backbone** of the business logic. They perform operations such as:

- **Voter Authentication & Validation**: Verifying user credentials and checking if the voter is registered and verified.
- **Vote Casting Logic**: Ensuring a voter can only vote once, validating election timing, and saving vote records securely.
- **Result Computation**: Counting votes, handling tie-breakers, and generating real-time results in a secure and efficient manner.

- **Audit Logging**: Recording every action performed during voting for transparency and future verification.

By separating business logic into services, the application maintains **loose coupling**, **reusability**, and **testability**—principles that are essential for long-term maintenance and scalability.

### 3. Data Transfer Objects (DTOs)

DTOs are used to **safely transfer data** between layers. For example, when a user casts a vote, a VoteDTO may be used to package the voter's ID, candidate ID, and election ID, reducing the risk of exposing sensitive data and preventing direct manipulation of entity objects.

### Spring Boot Advantages in the Voting System

Spring Boot brings several advantages to the development of the business logic layer, such as:

- **RESTful API Support**: Simplifies the creation of endpoints for frontend communication.
- **Security Integration**: Works seamlessly with Spring Security to implement login mechanisms, role-based access control, and encryption.
- **Dependency Injection**: Facilitates clean code and better modularization by decoupling object creation and usage.
- **Exception Handling**: Provides structured ways to handle errors like unauthorized access, invalid vote attempts, or system failures.
- **Transaction Management**: Ensures that database operations are atomic and consistent, which is critical in maintaining voting integrity

**Enforcing Election Integrity Through Business Logic**

The business logic layer ensures that the rules of the election process are strictly followed. This includes:

- Restricting voting access to authenticated and verified users.
- Preventing double voting or any form of manipulation.
- Dynamically validating election timing (votes only accepted during active periods).
- Validating admin actions to ensure only authorized personnel can alter elections or view sensitive data.
- Ensuring timely computation and publication of accurate results.

**3. Data Access Layer (Persistence):**

The data access layer utilizes the Java Persistence API (JPA) to interact with the underlying database. This layer is responsible for managing the persistence of data related to voters, elections, candidates, and votes. Entity classes, such as `Voter.java`, `Election.java`, and `Vote.java`, are mapped to their corresponding database tables, allowing for efficient data retrieval and manipulation. By abstracting the database interactions, this layer ensures that the business logic remains decoupled from the specifics of data storage, facilitating easier updates and modifications in the future.

This three-tier architecture ensures a clean separation of the user interface, business logic, and data storage, which not only simplifies testing and debugging but also allows for future expansion of the system. As the online voting system evolves, this modular design will enable the integration of new features, such as advanced voter verification methods or real-time result tracking, without disrupting the existing functionality.

# CHAPTER 7

# IMPLEMENTATION

## 7.1. Creating the Database

For the online voting system, the database design prioritizes simplicity and ease of testing. Instead of relying on an external database management system such as MySQL or SQLite, the application adopts an in-memory storage approach. This method allows all votingrelated data, including voter information, election details, candidates, and votes, to be hardcoded within a dedicated class file, such as `SampleData.java`. This class functions as a mock database, maintaining static lists of objects that encapsulate the necessary data for the voting process.

By utilizing this in-memory approach, the system significantly reduces setup complexity, making it portable and quick to deploy for small-scale use or demonstrations. This is particularly advantageous in environments where rapid prototyping is essential, such as during development phases or for proof-of-concept presentations. The in-memory storage allows developers to focus on functionality without the overhead of configuring and managing a traditional database system.

## 7.2. Connecting the Database to the Application

In this architecture, the application retrieves voting data by directly invoking methods from the `SampleData` class, rather than establishing a conventional database connection using JDBC. For instance, methods like `getVoters()`, `getElections()`, and `getCandidates()` return predefined lists of objects that represent voters, elections, and candidates, respectively. This data is then utilized by the user interface to display relevant information to the voters.

The absence of real-time data fetching ensures faster performance, as the application does not need to wait for database queries to execute. This design choice eliminates dependencies on network configurations or database server availability, making the system more resilient and easier to deploy in various environments. Such an integration

method is ideal for applications where persistent data storage is not required, allowing for quick iterations and testing of features without the complexities associated with traditional database management.

## 7.3. Creating the Main Window

The primary user interface of the online voting system is initialized through the `Main.java` class. Upon execution, this class creates an instance of the `VotingGUI` class, which is responsible for setting up the main voting window using Java Swing components. The window is designed to be user-friendly and visually organized, featuring a title panel that displays the election name, a candidate display area, and interactive buttons for casting votes.

The layout of the main window is responsive, ensuring that the application can be comfortably used on systems with various screen resolutions. This adaptability is crucial for providing a consistent user experience across different devices, whether voters are accessing the system from a desktop, tablet, or mobile device. The design emphasizes clarity and ease of navigation, allowing users to focus on the voting process without unnecessary distractions.

## 7.4. Displaying Frames Over the Main Window

The voting interface operates within a single main window, enhancing user experience by avoiding the creation of new windows or dialogs for each voting action. Instead, the application dynamically updates the content panel using Java Swing. This is achieved by modifying the visible components in response to user actions, such as when a voter selects a candidate and clicks the "Vote" button.

This approach improves performance and maintains a consistent visual flow, as users do not experience disruptive window pop-ups or context switches while casting their votes. By keeping the interaction within a single window, the application fosters a more engaging and streamlined voting experience, allowing voters to focus on their choices without unnecessary interruptions.

## 7.5. Processing Queries

User interaction is central to the online voting system. Each time a voter selects a candidate and clicks the "Vote" button, the application captures this input and processes it accordingly. The system validates the input by comparing the selected candidate against the list of available candidates stored in the corresponding data objects. This validation process ensures that the selected option is legitimate and updates the internal vote count for the chosen candidate.

The logic is structured to ensure that each vote is only counted once per voter, preserving the integrity of the electoral process and preventing any unintended manipulation of results. This careful handling of user input is essential for maintaining trust in the voting system, as it guarantees that each voter's choice is accurately reflected in the final tally.

## 7.6. Coding the Core Functionality

The core functionality of the online voting system is encapsulated within the GUI controller and data model classes. Key responsibilities of this core module include loading and storing the list of candidates, presenting one candidate at a time to the user, tracking the current voting state, accepting user input, and determining the correctness of the vote. Additionally, the system maintains a count of votes for each candidate and displays a final result summary upon the completion of the voting process.

Each of these functions is implemented in a modular and reusable manner, ensuring maintainability and ease of debugging. This modular design allows developers to make updates or enhancements to specific components without affecting the overall system, facilitating ongoing development and improvement of the voting application.

## 7.7. Handling User Inputs

User input is captured through Swing Action Listeners, which are associated with interactive components such as buttons. When a voter selects a candidate and clicks "Vote," the action listener triggers a series of events that validate the input, update the vote count, and provide feedback to the user. This event-driven architecture ensures a responsive user experience, allowing the application to react immediately to voter actions.

Real-time verification of user interactions is implemented to prevent unexpected behavior or data corruption. For instance, the application may disable the "Vote" button until a candidate is selected, ensuring that voters cannot proceed without making a choice. This proactive approach to user input handling enhances the overall reliability of the voting system.

## 7.8. Error Handling and Validation

Robust error handling mechanisms are integrated into the online voting system to ensure smooth and predictable application behavior. Key validation features include preventing users from proceeding without selecting a candidate, ensuring that navigation controls such as "Vote" are enabled or disabled appropriately based on the voting state, and gracefully managing the end-of-voting event by displaying a summary screen and disabling further interactions.

Additionally, the application employs try-catch blocks to handle potential runtime exceptions, such as null references or out-of-bounds errors, thereby avoiding abrupt program termination. This comprehensive approach to error handling not only enhances user experience but also contributes to the overall stability and reliability of the voting system.

# CHAPTER 8

# TESTING

## 8.1. Unit Testing

Unit testing is a critical phase in the software development lifecycle, focusing on the verification of individual units or components of a software system in isolation. In the context of the online voting system, unit testing primarily targeted foundational classes such as `Voter`, `Candidate`, and `Election`. These tests were conducted using the JUnit framework, which is widely recognized for its effectiveness in Java applications.

The primary goal of unit testing in this system was to ensure that the data models accurately stored and retrieved essential information, such as voter details, candidate names, and election dates. Key aspects tested included:

**Proper Initialization of Objects:** Each unit test verified that the constructors of the classes correctly initialized the objects with the expected values. For instance, when creating a `Voter` object, tests confirmed that the name, email, and verification status were set correctly.

**Accuracy of Getter Methods:** The tests assessed the functionality of getter methods, ensuring that they returned the correct values for each attribute. For example, the `getName()` method of the `Voter` class was tested to confirm it returned the voter's name as expected.

**Consistency of Return Values:** The tests also focused on methods that computed or returned specific values, such as the total number of votes for a candidate. Assertions were used to compare expected results with actual outputs, ensuring that the logic was functioning correctly.

**Verification of Edge Cases:** Special attention was given to edge cases, such as null inputs or empty data sets. Tests were designed to ensure that the application handled these scenarios gracefully, without throwing unexpected exceptions or producing incorrect results.

To enhance the testing process, mock objects were employed where necessary to simulate data responses. This approach allowed for isolated testing of components without relying on the actual implementation of other classes. Assertions were extensively used to compare expected versus actual results, providing a clear indication of any discrepancies.

The unit tests formed the foundation for identifying bugs early in the development process, ensuring that each component worked correctly before integration into the larger system. By validating the functionality of individual units, the team could confidently build upon a solid base, reducing the likelihood of issues arising during later stages of development.

## 8.2. Integration Testing

Integration testing aims to validate the interactions between multiple units or components of a system, ensuring that they work together as intended. For the online voting system, this phase involved testing how well the `SampleData` class interacted with the user interface managed by the `VotingGUI` class. The focus was on the seamless flow of data from the backend data structure to the front-end interface.

**Test scenarios included:**

**Verifying Data Loading:** One of the primary tests involved ensuring that all voters, candidates, and election data loaded from the `SampleData` class appeared correctly on the user interface. This was crucial for confirming that the application presented accurate information to voters.

**User Selections and Result Computation**: Integration tests also checked that user selections were passed accurately to the result computation logic. When a voter cast a

vote, the system needed to ensure that the correct candidate was identified and that the vote was counted appropriately.

**Handling Navigation Events:** The tests assessed how the interface reacted to navigation events, such as moving to the next candidate or submitting a vote. This included verifying that the application correctly updated the displayed information and maintained the state of the voting process.

**Invalid or Incomplete Inputs:** Special attention was given to how the interface handled invalid or incomplete inputs. For example, tests were conducted to ensure that the application prevented users from submitting a vote without selecting a candidate, thereby maintaining the integrity of the voting process.

The integration tests were semi-automated, supplemented by manual testing to confirm usability and user experience. Edge cases, such as navigating beyond the last candidate or attempting to submit a vote without making a selection, were specifically targeted to ensure that the application handled these scenarios gracefully.

By validating the interactions between components, integration testing played a crucial role in ensuring that the online voting system functioned as a cohesive unit. This phase helped identify any discrepancies in data flow and interaction, allowing for timely corrections before moving on to system testing.

## 8.3. System Testing

System testing involved evaluating the online voting application as a complete, functioning system. This phase ensured that the application met the requirements specified during the design phase and simulated real-world usage scenarios. The tests were designed to interact with all components of the application, validating the correctness of results and the overall user experience.

## Test cases focused on several key areas:

**Launching the Application:** The initial test case involved launching the application and verifying that all graphical user interface (GUI) elements rendered correctly. This included checking that buttons, labels, and input fields were displayed as intended.

**Navigating Through Candidate:** Tests were conducted to ensure that users could navigate through the list of candidates and submit their votes without encountering issues. This included verifying that the application correctly displayed the next candidate when prompted.

**Accurate Scoring:** The system was tested to ensure that scoring was accurate based on correct and incorrect responses. This involved simulating various voting scenarios and confirming that the final vote counts matched expectations**.**

**End-of-Voting Event Handling:** The application was evaluated for its ability to handle the end-of-voting event gracefully. This included displaying a summary screen with the results and disabling further interactions to prevent additional votes from being cast.

**Responsiveness Across Devices:** System testing also focused on verifying the application's responsiveness across different resolutions and screen sizes. This ensured that the voting interface remained user-friendly and accessible, regardless of the device used.

Stress testing was conducted by increasing the number of candidates and simulating rapid user interactions. The system was found to be stable, with no crashes or unexpected behaviors during extended usage. This phase of testing confirmed that the application could handle real-world scenarios effectively, providing a reliable platform for voters.

## Comprehensive Testing Approach

Overall, the comprehensive testing approach—combining unit, integration, and system testing—enabled robust validation of the online voting application's core features and user experience. Each phase contributed uniquely to ensuring reliability:

**Unit Tests:** Confirmed code-level correctness, allowing for early detection of bugs and ensuring that individual components functioned as intended.

Integration Tests: Ensured data consistency across modules, validating that components interacted correctly and that data flowed seamlessly from the backend to the frontend.

**System Tests:** Validated end-user behavior and edge case handling, simulating real-world usage to ensure that the application met user expectations and requirements.

In future iterations, the integration of automated GUI testing frameworks, such as Selenium or TestFX (for JavaFX), could be considered to facilitate regression testing and improve test coverage. These tools would allow for automated testing of the user interface, ensuring that changes to the application do not introduce new issues.

Moreover, integrating continuous integration (CI) tools such as Jenkins or GitHub Actions could streamline the testing pipeline, ensuring that changes to the codebase are automatically tested and validated. This would help maintain the integrity of the application over time, allowing for more efficient development and deployment processes.

# CHAPTER 9

## CODE IMPLEMENTATION

### 9.1 Login.jsp

```
<%@ page language="java" %>
<html>

<body>

   <div class="triangle orange"></div>
  <div class="triangle green"></div>

  <div class="container">
    <div class="header">
      <img src="images/Capture2.JPG" alt="Emblem" class="emblem">
      <h1><b><i>ONLINE VOTING SYSTEM<b><i></h1>


      <div class="user-icon"></div>
    </div>

    <img src="images/Capture.JPG" alt="Ashoka Chakra" class="chakra">

    <div class="login-box">
      <form action="login" method="post">
        <label>Username:</label>
        <input type="text" id="name" name="username" required>

        <label>Password:</label>
        <input type="password" id="password" name="password" required>

        <input type="submit" value="Login" />
      </form>
      <h1><b><i><marquee direction="left">Revolution at Your
Fingertip</marquee><b><i></h1>
    </div>
  </div>

</body>
<style>
    body {
      margin: 0;
      font-family: 'Segoe UI', sans-serif;
      background: white;
      position: relative;
      overflow: hidden;
    }
```

```
/* Triangular flag corners */
.triangle.orange {
  position: absolute;
  top: 0;
  left: 0;
  width: 0;
  height: 0;
  border-top: 500px solid #FF9933;
  border-right: 500px solid transparent;
  z-index: 1;
}

.triangle.green {
  position: absolute;
  bottom: 0;
  right: 0;
  width: 0;
  height: 0;
  border-bottom: 450px solid #138808;
  border-left: 450px solid transparent;
  z-index: 1;
}

.container {
  text-align: center;
  position: relative;
  z-index: 2;
}

.header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 20px;
  color : #000080;
  font-size: 30px
  background-image: linear-gradient(orange,white,green);
}

.emblem {
  width: 50px;
  height: 60px;
}

h1 {
  flex-grow: 1;
  text-align: center;
  font-family: 'Courier New', monospace;

  margin: 0;
}

.user-icon {
  font-size: 28px;
  margin-right: 10px;
}
```

```
    .chakra {
      width: 300px;
      opacity: 0.4;
      position: absolute;
      top: 50%;
      left: 50%;
      transform: translate(-50%, -50%);
      z-index: 0;
    }

    .login-box {
      width: 300px;
      margin: 100px auto;
      padding: 100px;
       top: 50%;
       left: 0.1%;
      text-align: center;
      background: rgba(255, 255, 255, 0.85);
      border-radius: 15px;
      position: relative;
      z-index: 2;
    }

    .login-box label {
      display: block;
      margin: 15px 0 5px;
      font-size: 14px;
      font-weight: bold;
    }

    .login-box input {
      width: 90%;
      padding: 10px;
      border-radius: 10px;
      border: 1px solid #ccc;
      margin-bottom: 10px;
    }

    .login-box button {
      background-color: #ff8040;
      border: none;
      padding: 10px 20px;
      border-radius: 10px;
      color: white;
      font-weight: bold;
      cursor: pointer;
      margin-top: 10px;
    }

    .login-box button:hover {
      background-color: #e67300;
    }
  </style>
</html>
```

## 9.2 Vote.jsp

```
<%@ page import="java.sql.*, com.vote.DBConnection" %>
<%@ page language="java" %>
<%
    Integer userId = (Integer) session.getAttribute("userId");
    if (userId == null) {
        response.sendRedirect("login.jsp");
        return;
    }
%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Cast Your Vote</title>
    <style>
        body {
            margin: 0;
            font-family: Arial, sans-serif;
            background-color: #f4f4f4;
        }

        header {
            background: linear-gradient(to right, #FF9933, white, #138808);
            color: navy;
            text-align: center;
            padding: 20px;
        }

        .container {
            max-width: 800px;
            margin: 30px auto;
            padding: 30px;
            background-color: white;
            border-radius: 15px;
            box-shadow: 0 0 15px rgba(0,0,0,0.1);
        }

        h2 {
            color: #138808;
            text-align: center;
            margin-bottom: 20px;
        }

        .candidate-option {
            background-color: #e6f2e6;
            border: 2px solid #138808;
            padding: 15px;
            margin-bottom: 10px;
            border-radius: 10px;
            font-size: 18px;
        }
```

```
        input[type="radio"] {
            transform: scale(1.3);
            margin-right: 10px;
        }

        .submit-btn {
            display: block;
            width: 100%;
            padding: 12px;
            background-color: #FF9933;
            color: white;
            border: none;
            border-radius: 8px;
            font-size: 18px;
            cursor: pointer;
            transition: background-color 0.3s ease;
        }

        .submit-btn:hover {
            background-color: #e67e00;
        }

        .footer-link {
            text-align: center;
            margin-top: 20px;
        }

        .footer-link a {
            text-decoration: none;
            color: #FF9933;
            font-weight: bold;
            border: 2px solid #FF9933;
            padding: 8px 15px;
            border-radius: 5px;
            transition: 0.3s;
        }

        .footer-link a:hover {
            background-color: #FF9933;
            color: white;
        }
    </style>
</head>
<body>

    <header>
        <h1>Welcome to the Voting Portal</h1>
        <p>Cast your vote with pride</p>
    </header>

    <div class="container">
        <h2>Cast Your Vote</h2>
        <form action="vote" method="post">
            <%
                try (Connection con = DBConnection.getConnection()) {
                Statement st = con.createStatement();
```

```
                    ResultSet rs = st.executeQuery("SELECT * FROM
candidates");
                    while (rs.next()) {
                        int cid = rs.getInt("id");
                        String name = rs.getString("name");
            %>
                <div class="candidate-option">
                    <label>
                        <input type="radio" name="candidate" value="<%= cid
%>" required />
                        <%= name %>
                    </label>
                </div>
            <%
                    }
                } catch (Exception e) {
                    e.printStackTrace();
                }
            %>
            <br>
            <input type="submit" class="submit-btn" value="Submit Vote" />
        </form>

        <div class="footer-link">
            <a href="login.jsp">EXIT</a>
        </div>
    </div>

</body>
</html>
```

## 9.3 home.jsp

```
<%@ page import="java.sql.*, com.vote.DBConnection" %>
<%@ page language="java" %>
<%
    Integer userId = (Integer) session.getAttribute("userId");
    if (userId == null) {
        response.sendRedirect("login.jsp");
        return;
    }
%>
<html>
<header>
    <h1>Welcome to the Voting Portal</h1>
    <p>Your vote, your voice - make it count!</p>
  </header>

  <div class="container">

    <div class="section">
      <h2>Current Poll</h2>
```

```
      <div class="poll-card">
        <img
src="https://upload.wikimedia.org/wikipedia/commons/thumb/8/81/TamilNadu_Logo.
svg/800px-TamilNadu_Logo.svg.png" alt="Poll Image">
        <div class="poll-info">
          <h3>Tamil Nadu Legislative Assembly Election</h3>
          <p>Ends on: 25 May 2026</p>
        </div>
        <a class="enter-btn" href="vote.jsp">Enter the polling</a>
      </div>
    </div>

    <div class="section">
      <h2>Upcoming Polls</h2>

      <div class="poll-card">
        <img
src="https://upload.wikimedia.org/wikipedia/commons/thumb/7/72/Election_Commis
sion_of_India_Logo.svg/250px-Election_Commission_of_India_Logo.svg.png"
alt="Poll Image">
        <div class="poll-info">
          <h3>Lok Sabha(Central election - PM)</h3>
          <p>Scheduled on: 20 June 2025</p>
        </div>
      </div>

      <div class="poll-card">
        <img
src="https://upload.wikimedia.org/wikipedia/commons/thumb/3/37/Emblem_of_Andhr
a_Pradesh.svg/330px-Emblem_of_Andhra_Pradesh.svg.png" alt="Poll Image">
        <div class="poll-info">
          <h3>Andhra Pradesh Legislative Assembly</h3>
          <p>Scheduled on: 1 August 2025</p>
        </div>
      </div>

    </div>

  </div>

  <footer>
    &copy; 2025 Indian Voting System | Made with pride
  </footer>
</body>
<head>
  <meta charset="UTF-8">
  <title>Voting System</title>
  <style>
    body {
      margin: 0;
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
    }

    header {
      background: linear-gradient(to right, #FF9933, white, #138808);
```

```
  color: navy;
  padding: 20px;
  text-align: center;
}

.container {
  padding: 30px;
  max-width: 1000px;
  margin: auto;
}

.section {
  background: white;
  border-left: 10px solid #FF9933;
  border-radius: 10px;
  margin-bottom: 30px;
  box-shadow: 0 4px 10px rgba(0,0,0,0.1);
  padding: 20px;
}

.section h2 {
  color: #138808;
}

.poll-card {
  padding: 15px;
  border: 2px solid #138808;
  border-radius: 8px;
  margin-top: 15px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  background-color: #e6f2e6;
}

.poll-card img {
  width: 80px;
  height: 80px;
  object-fit: cover;
  border-radius: 10px;
  margin-right: 20px;
}

.poll-info {
  flex: 1;
}

.poll-info h3 {
  margin: 0;
  color: #333;
}

.poll-info p {
  margin: 5px 0;
}
```

```
    .enter-btn {
      padding: 10px 20px;
      background-color: #FF9933;
      color: white;
      border: none;
      border-radius: 5px;
      text-decoration: none;
      font-weight: bold;
      transition: background-color 0.3s;
    }

    .enter-btn:hover {
      background-color: #e67e00;
    }

    footer {
      background-color: #138808;
      color: white;
      text-align: center;
      padding: 15px;
    }
  </style>
</html>
```

## 9.4 admin_dashboard.jsp

```
<%@ page import="java.sql.*, com.vote.DBConnection" %>
<%@ page language="java" %>
<%
    Integer userId = (Integer) session.getAttribute("userId");
    if (userId == null) {
        response.sendRedirect("login.jsp");
        return;
    }
%>
<html>
<header>
    <h1>Welcome to the Voting Portal</h1>
    <p>Your vote, your voice - make it count!</p>
  </header>

  <div class="container">

    <div class="section">
      <h2>Current Poll</h2>
      <div class="poll-card">
        <img
src="https://upload.wikimedia.org/wikipedia/commons/thumb/8/81/TamilNadu_Logo.
svg/800px-TamilNadu_Logo.svg.png" alt="Poll Image">
        <div class="poll-info">
          <h3>Tamil Nadu Legislative Assembly Election</h3>
          <p>Ends on: 25 May 2026</p>
```

```
        </div>
        <a class="enter-btn" href="vote.jsp">Enter the polling</a>
      </div>
    </div>

    <div class="section">
      <h2>Upcoming Polls</h2>

      <div class="poll-card">
        <img
src="https://upload.wikimedia.org/wikipedia/commons/thumb/7/72/Election_Commis
sion_of_India_Logo.svg/250px-Election_Commission_of_India_Logo.svg.png"
alt="Poll Image">
        <div class="poll-info">
          <h3>Lok Sabha(Central election - PM)</h3>
          <p>Scheduled on: 20 June 2025</p>
        </div>
      </div>

      <div class="poll-card">
        <img
src="https://upload.wikimedia.org/wikipedia/commons/thumb/3/37/Emblem_of_Andhr
a_Pradesh.svg/330px-Emblem_of_Andhra_Pradesh.svg.png" alt="Poll Image">
        <div class="poll-info">
          <h3>Andhra Pradesh Legislative Assembly</h3>
          <p>Scheduled on: 1 August 2025</p>
        </div>
      </div>

    </div>

  </div>

  <footer>
    &copy; 2025 Indian Voting System | Made with pride
  </footer>
</body>
<head>
  <meta charset="UTF-8">
  <title>Voting System</title>
  <style>
    body {
      margin: 0;
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
    }

    header {
      background: linear-gradient(to right, #FF9933, white, #138808);
      color: navy;
      padding: 20px;
      text-align: center;
    }

    .container {
      padding: 30px;
```

```
  max-width: 1000px;
  margin: auto;
}

.section {
  background: white;
  border-left: 10px solid #FF9933;
  border-radius: 10px;
  margin-bottom: 30px;
  box-shadow: 0 4px 10px rgba(0,0,0,0.1);
  padding: 20px;
}

.section h2 {
  color: #138808;
}

.poll-card {
  padding: 15px;
  border: 2px solid #138808;
  border-radius: 8px;
  margin-top: 15px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  background-color: #e6f2e6;
}

.poll-card img {
  width: 80px;
  height: 80px;
  object-fit: cover;
  border-radius: 10px;
  margin-right: 20px;
}

.poll-info {
  flex: 1;
}

.poll-info h3 {
  margin: 0;
  color: #333;
}

.poll-info p {
  margin: 5px 0;
}

.enter-btn {
  padding: 10px 20px;
  background-color: #FF9933;
  color: white;
  border: none;
  border-radius: 5px;
  text-decoration: none;
```

```
      font-weight: bold;
      transition: background-color 0.3s;
    }

    .enter-btn:hover {
      background-color: #e67e00;
    }

    footer {
      background-color: #138808;
      color: white;
      text-align: center;
      padding: 15px;
    }
  </style>
</html>
```

## 9.5 LoginServlet.java

```
package com.vote;


import java.io.IOException;

import java.sql.*;

import jakarta.servlet.*;

import jakarta.servlet.http.*;



public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse res)

            throws ServletException, IOException {


        String username = req.getParameter("username");

        String password = req.getParameter("password");
```

```
        try (Connection con = DBConnection.getConnection()) {
            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM users WHERE username=? AND password=?"
            );
            ps.setString(1, username);
            ps.setString(2, password);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                HttpSession session = req.getSession();
                session.setAttribute("userId", rs.getInt("id"));
                session.setAttribute("role", rs.getString("role"));
                session.setAttribute("username", username);

                // Redirect based on role
                if (rs.getString("role").equals("admin")) {
                    res.sendRedirect("admin_dashboard.jsp");
                } else {
                    res.sendRedirect("home.jsp");
                }
            } else {
                res.sendRedirect("login.jsp?error=Invalid+credentials");
            }
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

## 9.6 DBConnection

```
package com.vote;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/votingdb";
    private static final String USER = "root";
    private static final String PASSWORD = "jxt1201MySQL"; // ← Change this!

    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL, USER, PASSWORD);
```

```
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

**9.7 web.xml**

```
package com.vote;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/votingdb";
    private static final String USER = "root";
    private static final String PASSWORD = "jxt1201MySQL"; // ← Change this!

    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

# CHAPTER 10

# RESULTS

## 10.1 User Authentication Interface

The first point of interaction in the system is the login page, which provides a secure gateway for both voters and administrators. Each user must enter a valid username and password to proceed further. This feature ensures authentication and prevents unauthorized access to the system. The user interface is simple and user-friendly, designed to minimize login errors and streamline the process. The login functionality is a crucial step to differentiate roles—ensuring that voters access only the voting interface, while administrators are directed to a separate dashboard.

## 10.2 Voter Dashboard Displaying Poll Information

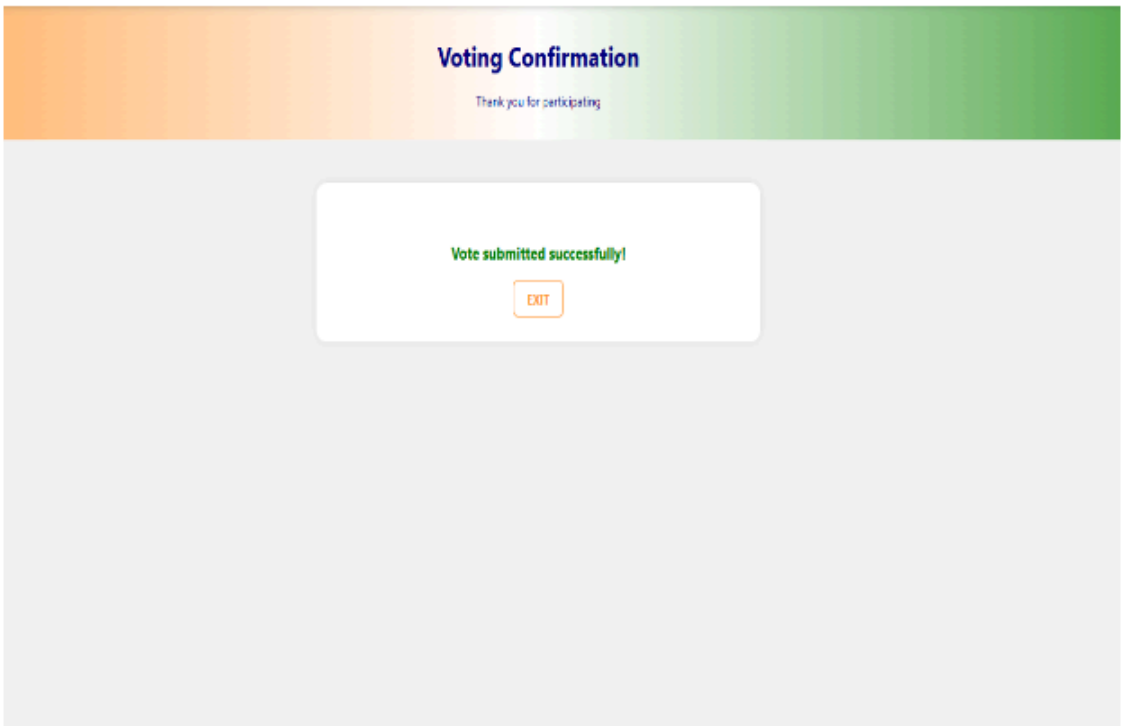
After a successful login, voters are taken to a dashboard where the details of current and upcoming elections are clearly displayed. The current poll section allows users to engage in ongoing elections, while the upcoming poll section helps them stay informed about future voting events. This organized layout improves voter awareness and participation by offering a structured overview. It reflects the system's ability to dynamically fetch and display real-time election data based on the logged-in user's credentials.

## 10.3 Voting Interface and Candidate Selection

Upon clicking on an active polling event, the system navigates the user to a voting interface that displays a list of all candidates contesting in that particular election. Each candidate is listed with an associated option (such as a radio button) allowing the user to select one of them. The voter can then confirm their selection and proceed to cast their vote. This module ensures a straightforward and intuitive voting process, designed to reduce confusion and eliminate accidental submissions. The interface design promotes ease of use, especially for first-time users.

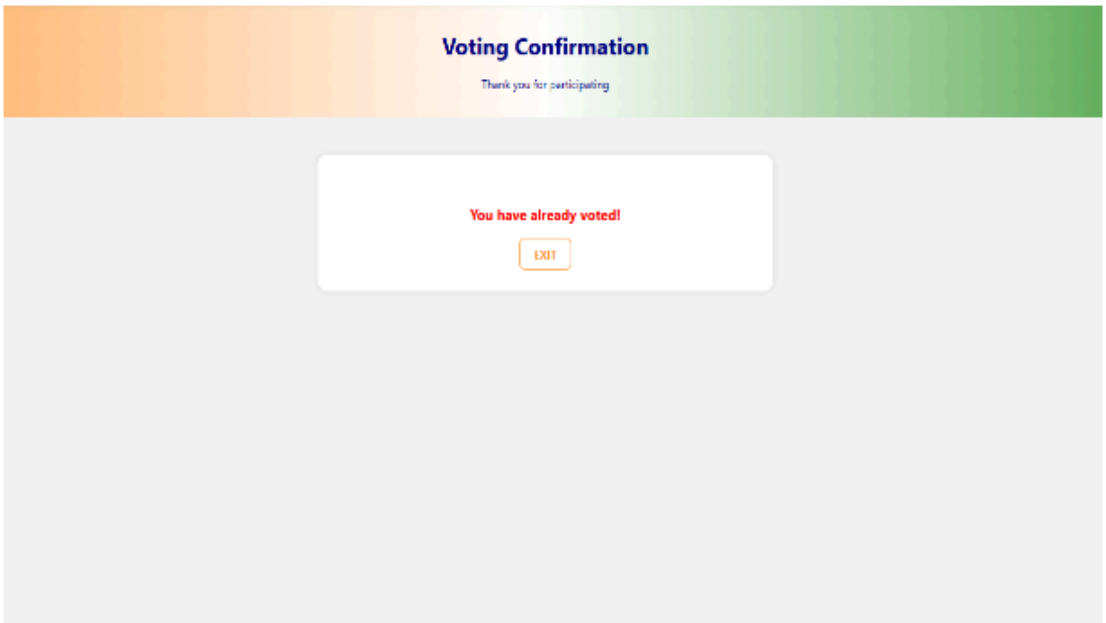
## 10.4 Vote Submission and Confirmation Message

Once a vote has been submitted, the system provides immediate feedback through a confirmation message that reads “Vote casted successfully.” This message not only confirms to the voter that their vote has been recorded but also signals the completion of the voting process. The display of such a message is important for user assurance and transparency. It reflects that the backend logic has executed properly—storing the vote, updating the database, and preventing any disruptions during submission.



## 10.5 Duplicate Vote Prevention Mechanism

To uphold election fairness and integrity, the system includes a critical validation step to check if the user has already cast a vote in the current election. If a voter attempts to vote again, the system displays a message stating, “You have already voted.” This ensures that each eligible voter can vote only once. This functionality not only prevents fraud but also demonstrates the successful implementation of a secure voting protocol, validating user activity against the database in real time.

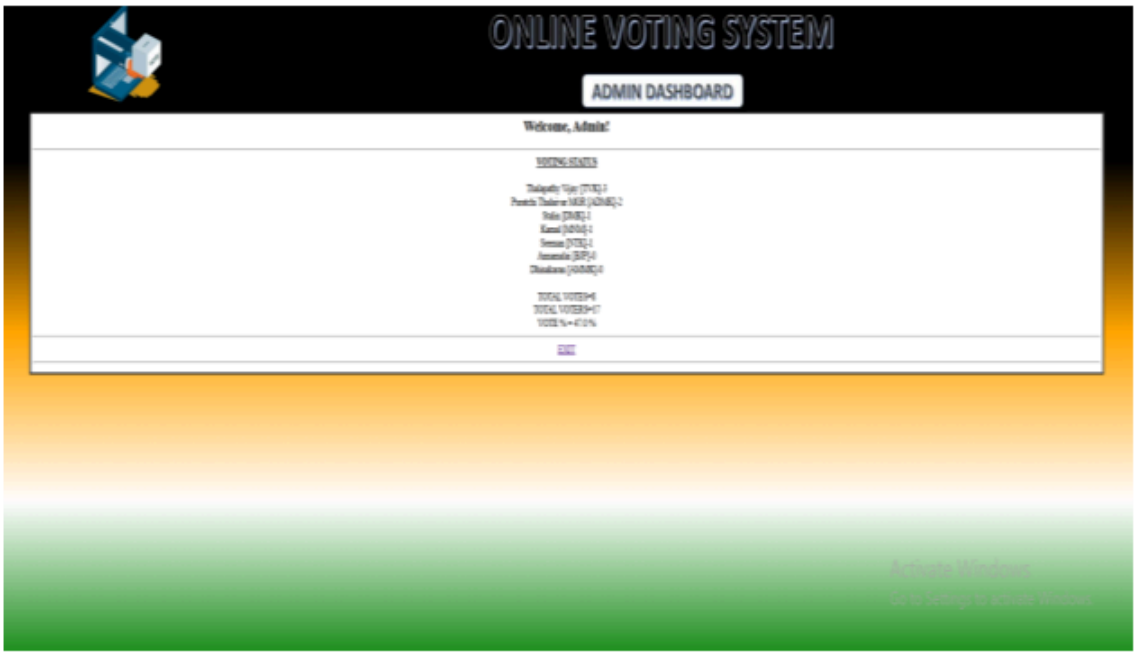


## 10.6 Administrator Login Functionality

In addition to the voter login, the system features a separate authentication mechanism for administrators. Admins use distinct credentials to access a more privileged interface. This separation of roles is essential in any secure online system, especially one handling sensitive electoral data. The admin login ensures that only authorized personnel can monitor voting activity, access real-time data, and manage poll-related tasks. This feature illustrates the effective use of role-based access control in the application.

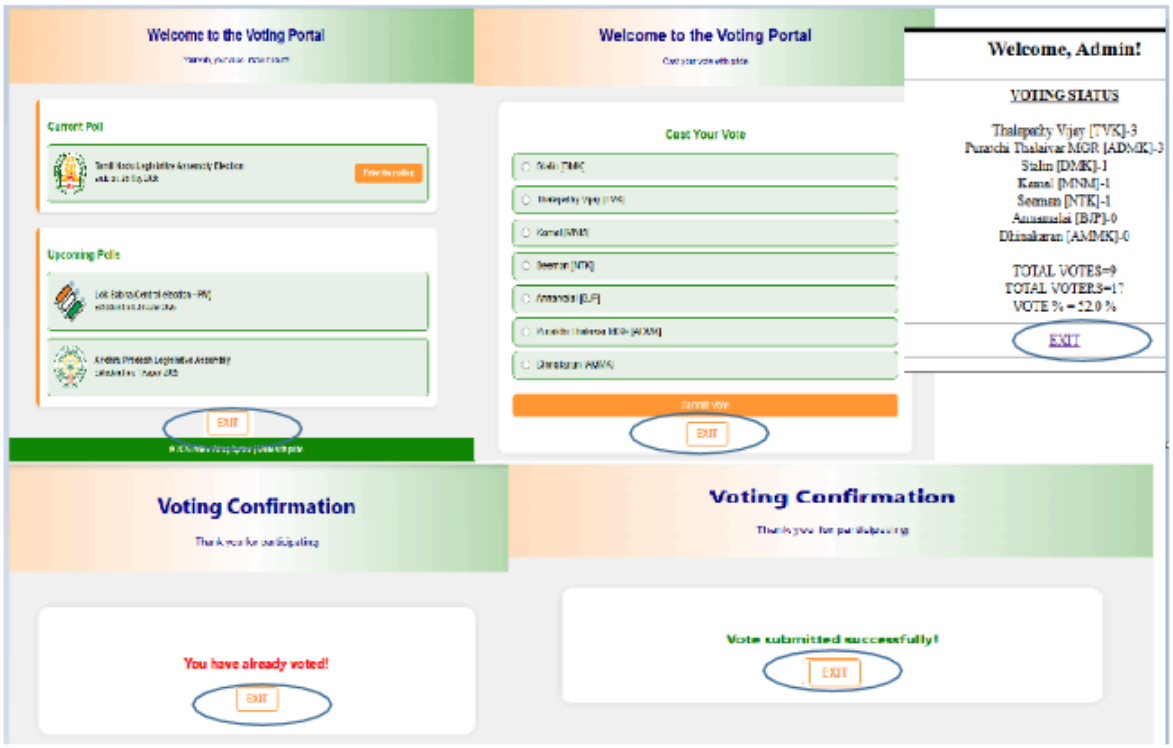
## 10.7 Admin Dashboard with Real-Time Voting Statistics

Once logged in, the administrator is redirected to a dedicated dashboard that provides a comprehensive overview of the current election. This includes the number of votes received by each candidate, the total number of votes cast in the election, the number of voters who have participated, and the percentage of voter turnout. All this data is displayed in real-time, allowing the admin to track and monitor the progress of the poll continuously. This feature highlights the system's ability to process and present live data accurately and efficiently, which is vital for transparency and administrative decision-making.



## 10.8 Exit Functionality and Secure Logout Mechanism

Every page in the system following the login screen is equipped with an exit button. When clicked, this button logs the user out and redirects them back to the login page. This feature provides a clean and user-friendly way to exit the session, which helps prevent unauthorized access if the system is left unattended. It also reflects good practice in session management and contributes to the overall security and usability of the system.



# CHAPTER 11

# CONCLUSION

The development of the online voting system represents a significant step toward modernizing electoral processes by leveraging technology to enhance accessibility, security, and efficiency. Throughout the project, a modular and layered architectural approach was adopted, enabling a clear separation of concerns among the user interface, business logic, and data management components. This separation not only facilitated organized development and testing but also established a strong foundation for scalability and future expansion.

A key achievement of the system lies in its user-centric design and responsiveness. By providing an intuitive and accessible interface, the application promotes voter engagement across diverse demographics and varying levels of technical proficiency. The strategic use of event-driven programming ensures that user inputs are processed swiftly and accurately, minimizing delays and creating a seamless voting experience. The design's responsiveness across various devices reaffirms its suitability for real-world electoral environments, where voters may access the platform through desktops, mobile devices, or tablets.

Robustness and reliability were prioritized through comprehensive testing at multiple levels, including unit, integration, and system testing. Each phase contributed uniquely to verifying the integrity of individual components, the smooth interaction among modules, and the overall functionality of the complete system. Testing scenarios were designed to reflect realistic voting conditions, including handling invalid inputs, managing navigation within the voting process, and enduring rapid user interactions. These efforts ensured that the system operates without critical failures or performance bottlenecks, thus bolstering trust in its security and stability.

The online voting system also exemplifies a balance between immediate functionality and long-term adaptability. Although designed with core voting features such as voter

authentication, candidate management, and secure vote casting, the architecture allows for the seamless integration of advanced features in future iterations. These enhancements may include persistent and scalable data storage, sophisticated authentication methods like biometrics, support for alternative voting schemes, and realtime analytics. Furthermore, plans to integrate automated testing and continuous integration pipelines underscore a commitment to maintaining code quality and accelerating development cycles.

Security considerations, while fundamental, are continuously evolving as new threats emerge. The system's implementation of data integrity controls and access restrictions provides a baseline level of protection. However, ongoing improvements, including encryption protocols, audit trails, and voter anonymity safeguards, remain essential to strengthen defenses against vulnerabilities and ensure compliance with regulatory standards.

Accessibility remains a core focus, reflecting the intent to democratize voting by enabling participation without barriers. Future enhancements aimed at mobile device compatibility and adherence to accessibility guidelines will empower a broader constituency to engage effectively with the platform. This emphasis aligns with a global trend toward inclusive digital transformation in democratic processes.

In summary, the online voting system successfully demonstrates how thoughtful design, rigorous testing, and a modular approach can deliver a dependable and scalable electronic voting platform. Its architecture not only supports current electoral needs but also provides a flexible framework that can adapt to emerging requirements and innovations. With continued development and strategic enhancements, this system has the potential to significantly contribute to more transparent, accessible, and efficient elections—ultimately strengthening democratic participation and trust.